💼 Hiring Quest – Frontend Angular Engineer (Mid-Level) @ Sahm Food

👋 We are **Sahm Food**, a technology-driven company operating in the food industry, building digital solutions that power restaurants, retail operations, and modern POS experiences.

Our products are used daily by cashiers, branch managers, and operations teams where speed, usability, and reliability directly impact business operations.

We're looking for a **Mid-Level Angular Engineer (3–5 Years Experience)** who enjoys solving complex UI problems, designing maintainable frontend architecture, and building production-quality applications—not just beautiful screens.

🕒 **Start Date:** Immediate
📍 **Location:** Nasr City, Cairo (On-Site)
💼 **Employment Type:** Full-Time
💰 **Salary:** 25,000 – 40,000 EGP

🛠️ How the Hiring Quest Works

1️⃣ Register for the quest

2️⃣ Receive the complete challenge after registration closes

3️⃣ Submit your solution before the deadline

4️⃣ Top candidates will be invited to a technical review session

5️⃣ One candidate will be hired, while other strong candidates may be considered for future openings

---

🔍 Who We're Looking For

✅ 3–5 years of professional Angular experience

✅ Strong TypeScript knowledge

✅ Solid understanding of Angular architecture and RxJS

✅ Experience building reusable component libraries

✅ Excellent state management skills

✅ Deep understanding of asynchronous programming

✅ Experience integrating REST APIs

✅ Strong HTML/CSS/SCSS skills

✅ Understanding of responsive and accessible UI

✅ Experience optimizing frontend performance

✅ Familiarity with testing Angular applications

✅ Strong Git workflow

✅ Ability to explain technical decisions clearly

🧠 Hiring Quest

Build a Smart Restaurant POS Dashboard

Business Context

Sahm Food operates hundreds of restaurants where every cashier works through a browser-based POS system.

The company is introducing a **new Smart Order Workspace** that combines traditional POS operations with AI-powered assistance.

The dashboard is used simultaneously by:

- Cashiers
- Branch Managers
- Kitchen Staff
- Customer Support

Unlike traditional CRUD dashboards, this workspace continuously receives live updates from multiple systems.

Your goal is **not** to build a complete application.

Instead, build a frontend architecture capable of handling complex UI interactions, asynchronous events, and scalable state management while keeping the application maintainable.

Core Modules

1. Live Orders Workspace

Display orders coming from different channels:

- Walk-in
- Delivery
- Online

Orders continuously change status.

Examples:

Received

↓

Preparing

↓

Ready

↓

Delivered

↓

Completed

Updates may arrive:

- Automatically
- From polling
- From simulated WebSocket events

The UI should remain responsive without unnecessary re-rendering.

---

2. AI Order Assistant

Each order includes an AI recommendation panel.

The assistant may suggest:

- Upselling items
- Allergy warnings
- Missing order information
- Delivery risks
- Kitchen overload warnings

AI responses arrive asynchronously.

They may:

- take several seconds
- fail
- retry
- stream partial responses (simulation)

The UI should gracefully handle every state.

---

3. Kitchen Load Monitor

Display live kitchen workload.

The workload affects order priorities.

When kitchen load changes:

Some orders become delayed.

Priority badges update automatically.

Recommendations change.

The UI should react without page refresh.

---

4. Advanced Product Search

Implement a search experience with:

- Instant filtering
- Debouncing
- Keyboard navigation
- Category filters
- Recent searches
- Highlighted matches

Large datasets should remain performant.

---

5. Offline Support

The application should tolerate temporary connection loss.

Examples:

Queue optimistic actions

Restore pending operations

Recover gracefully after reconnection

Prevent duplicated user actions

Architectural Expectations

Your project should demonstrate thoughtful frontend engineering.

Examples include:

- Feature-based architecture

- Smart separation between presentation and business logic
- Reusable components
- Proper Angular patterns
- Lazy loading where appropriate
- Scalable folder organization
- Proper typing
- Clean dependency boundaries

Avoid placing business logic directly inside components.

State Management

Choose any approach you believe is appropriate.

Examples:

- NgRx
- Signals
- Component Store
- Services with RxJS

Your choice matters less than your reasoning.

Be prepared to justify:

- Why you selected it
- Trade-offs
- Scalability considerations

UI/UX Expectations

We're evaluating engineering—not graphic design.

However, we expect:

- Clean UX
- Responsive layout
- Loading states
- Skeleton screens

- Error boundaries
- Empty states
- Smooth transitions
- Keyboard accessibility
- Accessible components
- Good information hierarchy

Fake Backend

You may simulate backend behavior.

Examples:

- Mock APIs
- JSON Server
- MSW
- Fake WebSocket
- RxJS timers
- Mock services

The quality of simulation is part of the evaluation.

Engineering Challenges

Your implementation should demonstrate solutions for problems such as:

- Multiple concurrent API requests
- Request cancellation
- Race conditions
- Optimistic updates
- Retry strategies
- Error recovery
- Caching
- Reactive programming
- Memory leak prevention
- Efficient change detection
- Preventing unnecessary component renders

Testing

Include meaningful automated tests.

At minimum, cover:

- State management behavior
- Search functionality
- Order status updates
- AI assistant state transitions
- Retry logic
- Offline synchronization
- Component behavior
- Critical business flows

Unit tests are expected. Integration tests are a plus.

Documentation

Include a README describing:

- Project architecture
- Folder structure
- Design decisions
- State management approach
- Performance optimizations
- Assumptions
- Known limitations
- Future improvements

Deliverables

Submit a GitHub repository containing:

- Complete source code

- README
- Installation instructions
- Environment configuration
- Tests
- Mock backend implementation

Additionally include:

- Postman or Bruno collection (if mock APIs are exposed)
- Any mock datasets used
- Architecture diagram (optional but recommended)

📹 Technical Walkthrough Video (Required)

Record a **12–20 minute** screen recording.

The video should include:

Part 1 — Product Demo

Demonstrate:

- Live order updates
- AI assistant behavior
- Search functionality
- Offline recovery
- Kitchen load updates
- Error handling
- Loading states

Part 2 — Engineering Deep Dive

Explain:

- Overall architecture
- Why you structured the project this way
- State management decisions
- Component communication strategy

- RxJS usage
- Performance optimizations
- Change detection strategy
- Lazy loading
- Reusability decisions
- Error handling
- Testing strategy

Part 3 — Live Code Navigation

Walk through your codebase and explain:

- Folder organization
- Shared components
- Services
- State layer
- API abstraction
- Reusable utilities
- Custom directives/pipes (if any)

Part 4 — Trade-offs

Discuss:

- What you intentionally simplified
- What you would improve with more time
- Technical debt you accepted
- Alternative architectural approaches you considered
- How this project could scale to hundreds of screens

🤖 AI Usage Policy

AI tools are allowed.

However, this challenge evaluates **your engineering ability**, not your ability to generate code.

If you used AI tools, your submission **must include**:

- Which AI tools were used
- The main prompts or workflows you relied on
- Relevant conversation excerpts or prompt history
- Which parts were AI-generated
- Which parts you designed or significantly modified yourself
- Architectural decisions you personally made
- How you verified AI-generated code
- Any incorrect AI suggestions you rejected and why

Failure to disclose AI usage may negatively affect the evaluation.

📋 Evaluation Criteria

Candidates will be evaluated on:

- Angular architecture
- TypeScript quality
- RxJS proficiency
- State management
- Performance optimization
- Component design
- Code maintainability
- Scalability
- Testing quality
- Accessibility
- Error handling
- Documentation
- Communication during the walkthrough
- Engineering reasoning
- Overall frontend craftsmanship
